Google Summer of Code 2026 – API Dash

1. Project Information

Project Title: Agentic AI Powered Self-Healing API Debugging and Optimization System

Project Summary:

API debugging is currently a manual and time-consuming process where developers must repeatedly inspect responses, identify errors, and modify requests to achieve a working result. This leads to inefficiencies, especially when dealing with complex APIs or authentication and schema-related issues.

This project proposes an intelligent debugging system within API Dash that combines rule-based validation with agentic AI workflows to automatically analyze API failures, suggest fixes, and iteratively improve requests. The system introduces a self-healing mechanism that can modify parameters, retry execution in a controlled environment, and provide optimized configurations.

By integrating debugging intelligence, self-healing workflows, and performance insights directly into API Dash, this project aims to transform API Dash from a request-testing tool into an intelligent, AI-native developer assistant that actively helps developers debug, fix, and optimize APIs in real-time.

Prototype Github Link : https://github.com/Greeva48/api-self-healing-debugger

2. Contributor Information:

Name: Greeva Dineshkumar Patel
Resume: Greeva_Patel_Resume
Email: greevapatel04@gmail.com

GitHub: https://github.com/Greeva48
LinkedIn: https://www.linkedin.com/in/greeva-patel-1a7b11299/
Discord: greevapatel (FOSS Developers Group / API Dash Discord)

- Username: greevapatel
- Link: https://discord.com/users/762692760049549323

Timezone: IST (UTC +5:30)

3. Background & Experience:

I am a backend-focused software engineer with strong experience in building scalable systems and API-driven platforms.

I worked as a Software Engineer at ISRO (Space Applications Centre), where I designed and developed over 30+ REST APIs and built high-throughput asynchronous pipelines handling more than 10,000 records per batch. I also designed relational database schemas with indexing, role-based access control (RBAC), and audit logging to ensure reliability and security.

Through this experience, I developed a deep understanding of API architecture, debugging challenges, performance bottlenecks, and system scalability. I worked on optimizing API latency, handling failures gracefully, and designing systems that can operate reliably in production environments.

In addition to backend systems, I have experience working with AI/ML pipelines, including building models for data processing and anomaly detection. This combination of backend engineering and AI enables me to design intelligent systems that are both scalable and practical.

I am particularly interested in improving developer experience through intelligent tooling, and this project aligns closely with my experience in API systems and system design.

4. Problem Statement:

API debugging today is largely a manual and repetitive process. Developers must inspect responses, identify issues such as authentication failures, malformed payloads, or incorrect parameters, and iteratively modify requests until a successful result is achieved.

Existing API tools primarily focus on request execution and response visualization but lack intelligent mechanisms to analyze failures or guide developers toward fixes. This becomes increasingly inefficient when working with complex APIs, dynamic schemas, or AI-based endpoints.

As a result, developers spend significant time debugging issues that could be identified and resolved automatically. There is a clear need for an intelligent system that can detect errors, suggest corrections, and assist in improving API reliability and performance.

5. Proposed Solution:

To address the limitations of existing API debugging workflows, this project proposes an intelligent, modular debugging and optimization system integrated within API Dash.

The system combines rule-based validation with agentic AI workflows to automatically analyze API failures, suggest corrections, and iteratively improve request configurations. It introduces a self-healing mechanism where API requests can be modified, retried, and optimized in a controlled environment.

Instead of relying solely on manual debugging, developers will be assisted by an intelligent system capable of detecting issues, recommending fixes, and improving API performance. This approach enhances developer productivity, reduces debugging time, and improves overall API reliability.

The detailed architecture and implementation of this system are described in the following section.

6. System Design:

Architecture Overview

The proposed system follows a modular pipeline architecture where API requests are analyzed, validated, and iteratively improved using both rule-based and AI-driven components. Each stage of the pipeline is designed to handle a specific responsibility, ensuring scalability, maintainability, and efficient processing of API debugging workflows.

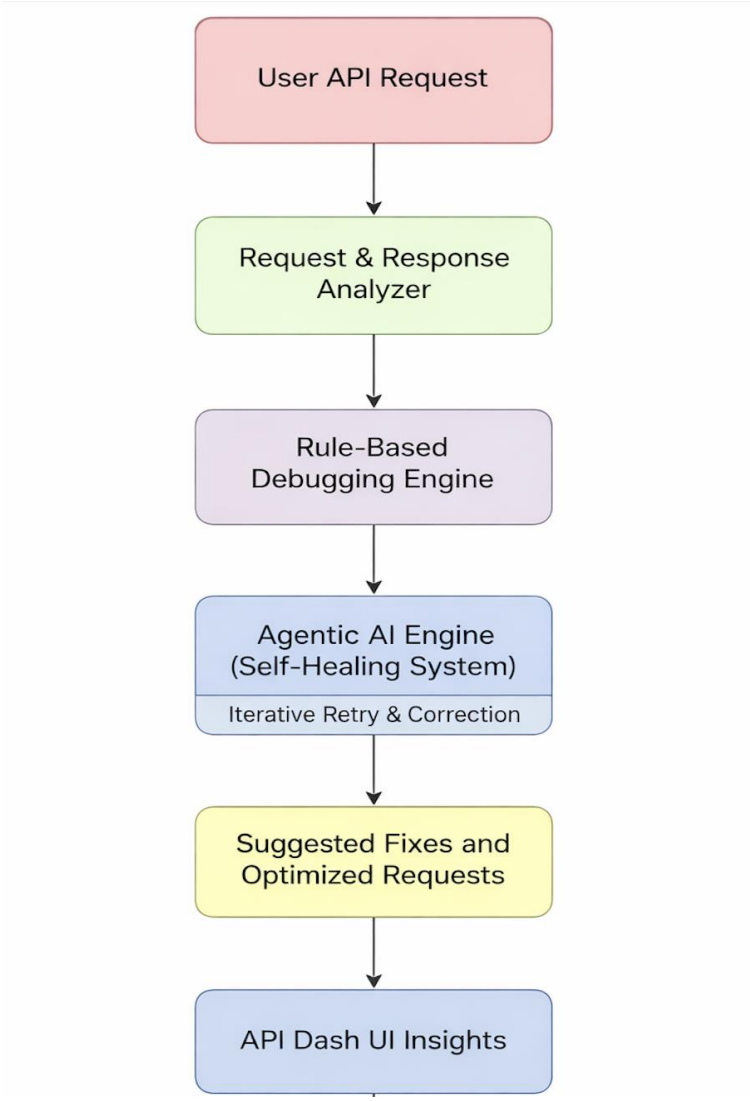


Figure 1: Agentic API Debugging and Optimization Workflow

Component Breakdown:

i.Request & Response Analyzer:

This component processes API inputs and outputs, extracting key information such as headers, payload structure, and response status. It acts as the entry point for identifying potential issues.

ii.Rule-Based Debugging Engine:

This layer performs deterministic validation checks to identify common errors. It ensures quick and reliable detection of issues before invoking more computationally intensive AI reasoning.

iii.Agentic AI Engine (Self-Healing System):

The core intelligence layer analyzes errors, modifies request configurations, and performs iterative retries. It identifies optimal fixes by evaluating multiple variations of the request in a controlled environment.

iv.Optimization Engine:

This module evaluates API performance metrics such as latency and payload size. It provides recommendations to improve efficiency, including caching, batching, and retry strategies.

v.UI Insights Layer:

All debugging results, suggestions, and optimizations are presented through the API Dash interface, enabling developers to easily understand and apply improvements.

7.Methodology / Implementation Approach:

The implementation of the proposed system will follow a staged and modular approach to ensure reliability, scalability, and ease of integration with the existing API Dash architecture.

i. Rule-Based Validation First Approach

The system will initially apply deterministic rule-based checks to identify common API issues such as missing headers, invalid payload structures, incorrect endpoints, and

authentication failures. This ensures fast and reliable error detection while reducing unnecessary dependency on AI models.

ii. Controlled AI Invocation

The Agentic AI Engine will be triggered only when rule-based validation is insufficient. The AI component will analyze the request-response pair and generate potential fixes based on error patterns, context, and API structure. This hybrid approach minimizes hallucination and improves accuracy.

iii. Iterative Retry Mechanism

The system will implement a bounded retry loop where modified API requests are executed in a controlled environment. Each iteration will be evaluated based on response success, error reduction, and consistency. A maximum retry limit will be enforced to prevent infinite loops.

iv. Context-Aware Request Modification

The AI engine will modify specific parts of the request such as headers, body parameters, and query values instead of regenerating entire requests. This ensures minimal changes and preserves the original developer intent.

v. Performance Evaluation and Optimization

Once a valid response is achieved, the system will analyze performance metrics such as latency, payload size, and response patterns. Based on this, optimization suggestions such as caching, batching, and retry strategies will be generated.

vi. Integration with API Dash

All components will be designed as modular services that can be integrated into API Dash's existing architecture. The debugging insights and suggestions will be surfaced through the UI for easy interaction.

This methodology ensures a balance between deterministic validation and intelligent reasoning, enabling a practical and scalable self-healing API debugging system.

8.Tech Stack:

Backend: Node.js / Python (FastAPI) for API processing and debugging logic

AI/ML: LLM-based reasoning (OpenAI / open-source models) for error analysis and suggestions

Frontend Integration: React / Flutter (API Dash UI integration)

Database: PostgreSQL / SQLite for storing debugging patterns and logs

DevOps: Docker for containerized execution and controlled retry environments

9. Challenges & Mitigation:

While building an intelligent API debugging system, several challenges may arise. The following approaches will be used to address them:

i. AI Hallucination and Incorrect Suggestions

AI models may generate inaccurate or irrelevant fixes. To mitigate this, the system will rely on rule-based validation as the first layer and use AI only when necessary. Additionally, responses will be validated before being suggested to the user.

ii. Infinite Retry Loops

The agentic system may repeatedly attempt fixes without reaching a valid solution. To prevent this, a maximum retry limit will be enforced, and the system will require user intervention after a defined number of attempts.

iii. Handling Complex or Large API Responses

Processing large responses can impact performance. This will be addressed using asynchronous processing and selective parsing to focus only on relevant parts of the response.

iv. Performance Overhead

Integrating AI-based reasoning may introduce latency. To mitigate this, caching mechanisms will be implemented for repeated error patterns, and lightweight models or optimized prompts will be used.

v. Generalization Across Different APIs

APIs vary widely in structure and behavior. The system will be designed to be modular and adaptable, allowing it to handle diverse API patterns while maintaining accuracy.

10. Prototype / Feasibility:

To validate the feasibility of the proposed approach, I implemented a working prototype that demonstrates rule-based API debugging and basic fix suggestions.

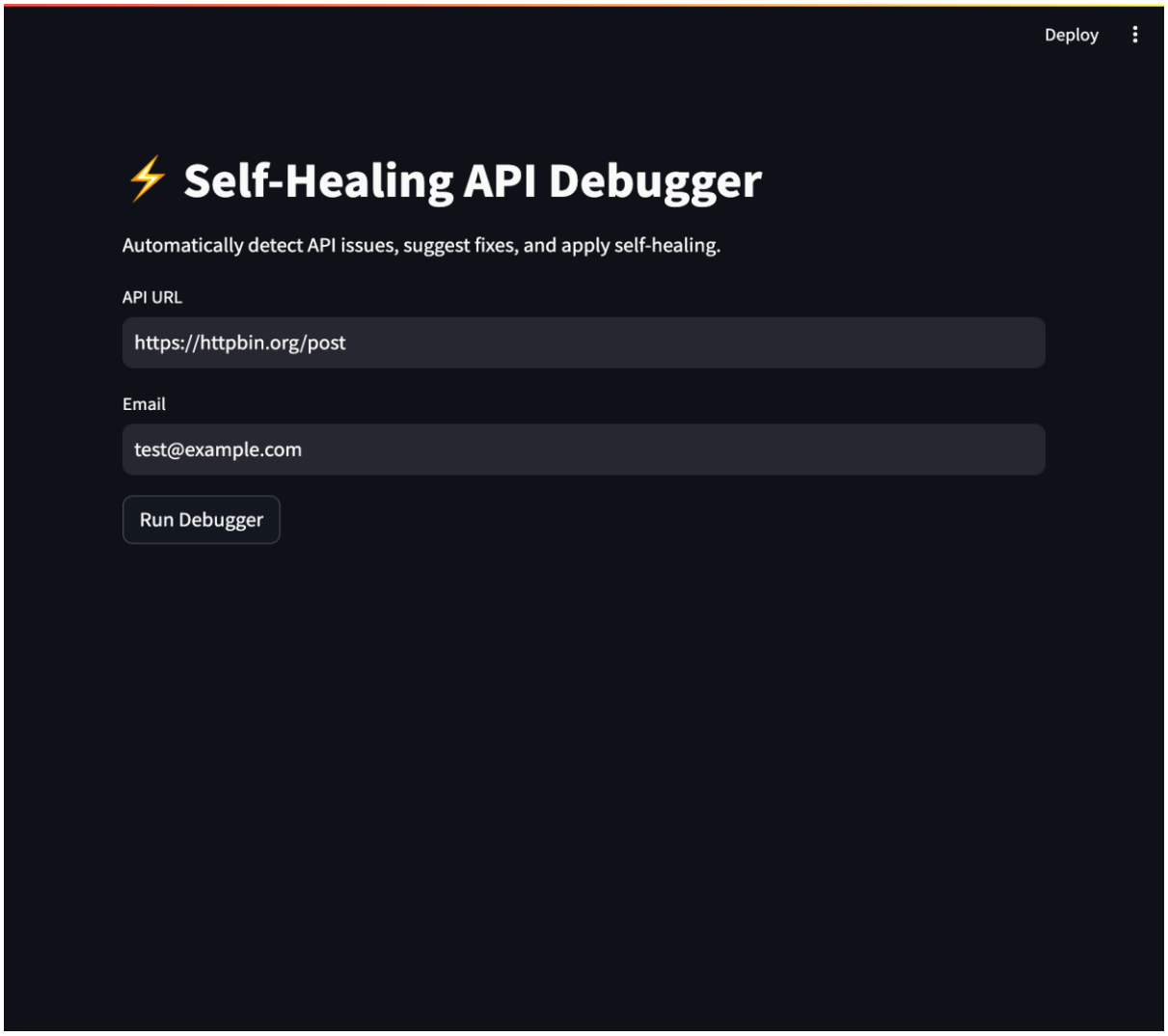
The prototype analyzes API requests, detects issues such as missing parameters or invalid structures, and provides actionable corrections. It further demonstrates a self-healing workflow where requests are automatically modified and retried to achieve a successful response.

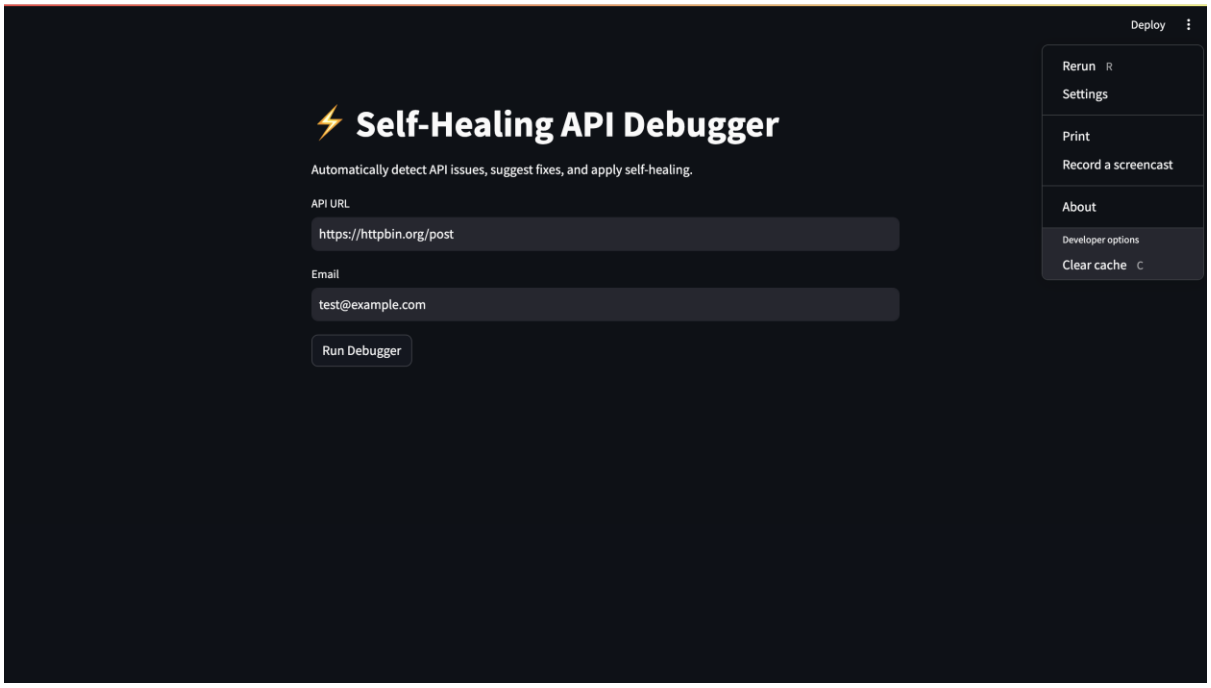
This end-to-end flow from issue detection to automatic correction and successful execution, validates the practicality of building an intelligent, self-healing API debugging system.

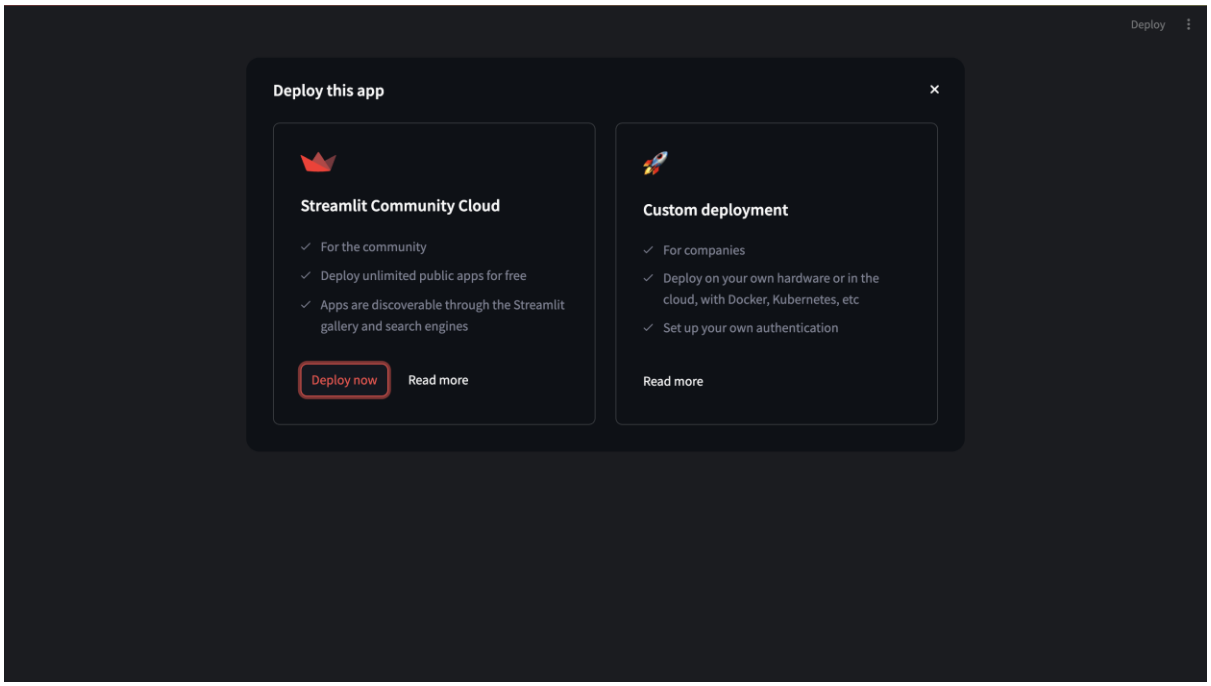
This prototype also serves as a strong foundation for integrating AI-driven reasoning, iterative optimization, and deeper debugging capabilities in the complete system.

Prototype Github Link : https://github.com/Greeva48/api-self-healing-debugger

SCREENSHOTS OF PROTOTYPE PROJECT:




Deploy
Rerun R
Settings
Print
Record a screencast
About
Developer options
Clear cache C
⚡ Self-Healing API Debugger
Automatically detect API issues, suggest fixes, and apply self-healing.
API URL
https://httpbin.org/post
Email
test@example.com
Run Debugger


Deploy
Deploy this app
Streamlit Community Cloud
For the community
Deploy unlimited public apps for free
Apps are discoverable through the Streamlit gallery and search engines
Deploy now
Read more
Custom deployment
For companies
Deploy on your own hardware or in the cloud, with Docker, Kubernetes, etc
Set up your own authentication
Read more

Deploy

⚡ Self-Healing API Debugger

Automatically detect API issues, suggest fixes, and apply self-healing.

API URL

https://httpbin.org/post

Email

test@example.com

Run Debugger

Initial Request


```
{
  "email" : "test@example.com"
}
```


Detected Issues

Missing required fields: password

Add fields: password

AI Suggestions

AI Suggestion: Add fields: password

Deploy

Healed Request


```
{
  "email" : "test@example.com"
  "password" : "default_password"
}
```


Final Response

Status: 200


```
{
  "args" : {}
  "data" : "{"email": "test@example.com", "password": "default_password"}"
  "files" : {}
  "form" : {}
  "headers" : {
    "Accept" : "*/*"
    "Accept-Encoding" : "gzip, deflate, br, zstd"
    "Content-Length" : "61"
    "Content-Type" : "application/json"
    "Host" : "httpbin.org"
    "User-Agent" : "python-requests/2.32.2"
    "X-Amzn-Trace-Id" : "Root=1-69cbeded-77f00cb646b8f7dd6cbbb0f3"
  }
  "json" : {
    "email" : "test@example.com"
    "password" : "default_password"
```

11. Timeline:

The project will be executed over a 12-week period with a structured and incremental development approach.

Community Bonding Period (Before Coding Phase)

- Understand API Dash codebase and architecture
- Discuss requirements and finalize system design
- Set up development environment and tools

Weeks 1–2

- Implement Request and Response Analyzer
- Parse API inputs/outputs (headers, payload, status codes)
- Identify common structural issues

Weeks 3–4

- Develop Rule-Based Debugging Engine
- Implement validation checks for common API errors
- Test on different API request scenarios

Weeks 5–6

- Develop Agentic AI Engine
- Implement AI-based error analysis and fix suggestions
- Design prompt structure and response handling

Weeks 7–8

- Implement Iterative Retry Mechanism
- Enable controlled request modification and re-execution

- Evaluate response improvements across iterations

Weeks 9–10

- Develop Optimization Engine
- Analyze performance metrics (latency, payload size)
- Generate optimization suggestions (caching, batching, retries)

Week 11

- Integrate all components with API Dash UI
- Display debugging insights and suggestions
- Perform end-to-end testing

Week 12

- Bug fixing and performance optimization
- Final documentation and code cleanup
- Prepare final submission

I will maintain consistent communication with mentors throughout the project to ensure alignment with goals and timely progress. Weekly check-ins will be conducted to discuss updates, challenges, and feedback.

All development progress will be tracked through structured GitHub commits and pull requests, enabling transparency and continuous review. I will actively incorporate mentor feedback into each development phase and iterate on improvements accordingly.

Additionally, I will remain available for discussions on Discord or other communication channels as preferred by the mentors, ensuring quick resolution of blockers and efficient collaboration.

Phase	Duration	Work
Community Bonding	Before Coding Phase	Understand API Dash architecture, finalize system design, setup development environment, discuss requirements with mentors
Phase 1	Weeks 1–2	Implement Request & Response Analyzer, parse API inputs/outputs, detect structural issues
Phase 2	Weeks 3–4	Develop Rule-Based Debugging Engine, implement validation checks, test on multiple API scenarios
Phase 3	Weeks 5–6	Develop Agentic AI Engine, implement error analysis and fix suggestions, design prompt handling
Phase 4	Weeks 7–8	Implement Iterative Retry Mechanism, enable controlled request modification and re-execution
Phase 5	Weeks 9–10	Build Optimization Engine, analyze performance metrics, generate optimization suggestions
Phase 6	Week 11	Integrate all components with API Dash UI, display debugging insights, perform end-to-end testing
Final Phase	Week 12	Bug fixing, performance optimization, documentation, final submission

12.Expected Outcomes:

The proposed system will significantly enhance API Dash by introducing intelligent debugging and optimization capabilities.

Developers will be able to automatically detect API issues, receive actionable fix suggestions, and iteratively improve request configurations through a self-healing workflow. This will reduce manual debugging effort and improve overall developer productivity.

The system will also provide performance insights, enabling developers to optimize API usage through recommendations such as caching, batching, and retry strategies.

Additionally, the integration of agentic AI workflows will position API Dash as a next-generation AI-powered developer tool, aligned with emerging trends such as MCP and intelligent automation.

13. Why Me / Conclusion:

I am a strong fit for this project due to my experience in building scalable backend systems and API-driven platforms.

During my time at ISRO (Space Applications Centre), I designed and deployed 30+ REST APIs and built high-throughput asynchronous pipelines handling large-scale data. This experience gave me a deep understanding of API architecture, debugging challenges, and system reliability in production environments.

In addition, I have experience working with AI/ML systems, enabling me to design intelligent solutions that combine rule-based validation with AI-driven reasoning.

My background in system design, distributed systems, and performance optimization aligns closely with the requirements of this project. I am confident in my ability to design and deliver a practical, scalable, and impactful solution within the GSoC timeline.

I have also built a working prototype for this idea, demonstrating my ability to translate concepts into real, usable systems and validating the feasibility of the proposed approach.

I am highly motivated to contribute to API Dash and collaborate closely with mentors to build an intelligent system that significantly improves developer experience.